

Craig Dilliplane

craig.t.dilliplane@gmail.com | +1 (609) 287-5572 | New York, NY

Representative Software Contributions

1. Differentiable APIC-DFSPH Solver Blending

Context and role: I co-developed a research prototype for learned coupling between two fluid solvers (APIC and DFSPH). At each time step, both solvers produced candidate particle velocities, and a neural gate predicted per-particle blending weights to interpolate continuously between the updates. I implemented and debugged Taichi/Python components for particle-grid transfer, SPH-style neighbor search, obstacle/boundary handling, adaptive targets, loss terms, diagnostics, and comparison of per-particle MLP and PointNet-style gates.

Technical problem: The project addressed how to combine two fluid solvers with complementary strengths without relying on hard-coded switching rules. APIC provides stable particle-grid updates and coherent large-scale motion, while DFSPH preserves particle-level transport and enforces incompressibility behavior near boundaries and obstacles. The software challenge was to make this solver selection continuous, trainable, and interpretable: both solvers needed to run at each timestep, a neural gate needed to learn per-particle blending weights, and the resulting coupled system needed physics-informed targets and losses that could be evaluated through simulation rollouts without introducing unstable or nonphysical behavior.

Implementation, validation, and reflection: Features combined particle geometry, tunnel-relative state, and velocity statistics. Supervision used online physics-informed targets based on density, boundary/obstacle proximity, lookahead obstacle interaction, teacher-blended velocities, and commitment regularization, with an exploratory divergence-loss term. I evaluated training loss, learned-vs-target gates, gate histograms, velocity statistics, and rollout snapshots comparing blended, APIC, and DFSPH behavior. I would next add explicit local-neighborhood context, clearer benchmark scenes, and more systematic tests of velocity-level and divergence losses.

2. Automated Simulation Grading and Validation Infrastructure

Context and role: As part of my computer animation teaching and grading work, I developed Python infrastructure for running, validating, and comparing numerical simulation submissions against expected behavior. I worked on the execution workflow, submission-structure validation, numerical comparison logic, debugging, and failure classification.

Technical problem: Simulation submissions needed to follow a prescribed folder and file structure before they could be executed reliably, and successful execution alone was not enough to determine correctness. A submission could import and run while producing malformed state data, timing out, or diverging numerically over time. The tool therefore needed to validate the submission layout and data contract, execute simulations in a controlled way, and distinguish structural errors, import errors, runtime failures, malformed outputs, timeouts, and numerical divergence.

Implementation, validation, and reflection: I structured the workflow around staged validation: checking that each submission matched the expected directory and file structure, running the simulation pipeline, extracting comparable state data, and comparing outputs against an oracle simulation that I built and verified through test cases. I wrote validators for expected structure and output format, added tolerance-based numerical diagnostics, and used staged reporting so failures could be traced to the appropriate point in the pipeline. I tested the tool against known-good submissions, intentionally broken submissions, malformed folder structures, timeout cases, and simulations designed to diverge from the oracle. If revisiting the project, I would formalize regression tests for each failure category earlier, document the submission data contract more clearly, and add automated CI checks, so that changes to parsing, timeout handling, tolerance thresholds, or oracle comparison logic would not silently alter grading behavior.

3. Primordial Black Hole Merger Signal Modeling Workflow

Context and role: As part of my cosmological research work, I developed computational workflows for estimating potentially observable gravitational-wave signatures from primordial black hole merger scenarios. The code explored how constituent black hole sizes, relative proximity, and orientation affected predicted frequency thresholds and signal-strength estimates. I worked on translating general-relativistic expressions and statistical significance criteria into reproducible numerical calculations of amplitude/frequency, parameter sweeps, and analysis outputs.

Technical problem: The project addressed how to turn a theoretical merger model into an inspectable computational workflow for future-state gravitational wave detectors. The software needed to evaluate signal behavior across a range of black hole sizes, angular configurations, and separation/proximity assumptions, then produce comparable estimates of frequency thresholds and signal strength. Rather than treating each calculation as a one-off script, the workflow needed to preserve the physical assumptions behind each run and make the dependence on input parameters clear.

Implementation, validation, and reflection: I implemented scripts for numerical evaluation, parameter-space exploration, and visualization of predicted amplitude/frequency signal behavior as functions of black hole size, orientation, and proximity. I validated outputs by checking limiting cases, comparing trends across parameter sweeps, and inspecting whether the results were physically consistent with the underlying general-relativistic model and statistical assumptions. If revisiting the project, I would separate model definitions from plotting and analysis code more cleanly, add regression tests for known reference cases, and document the assumptions behind each computational experiment more explicitly.